Week 8 Asynchronous Materials Guide

For Loops

This Week's Topics Overview

Building on everything you've learned about lists, conditionals, and strings, you'll now learn to:

- Use for loops to iterate through lists and sequences
- Generate number sequences with the range() function
- Loop through strings character by character
- Create nested loops for complex patterns
- Apply common loop patterns: accumulating, counting, filtering
- Combine loops with conditionals for powerful data processing

Start Here: Video Tutorial

Programming with Mosh - For Loops

- **Duration:** 14 minutes (For Loops section)
- **What you will learn:** For loops, range(), iterating through sequences
- **Why this helps:** Clear, concise explanations with practical examples
- **Study approach:** Pause and code along with each example
- **Note:** Focus on the for loops section of this tutorial

Core Resources: For Loops

W3Schools Python For Loops

https://www.w3schools.com/python/python_for_loops.asp

- Basic for loop syntax and examples
- Interactive "Try it Yourself" examples

- Simple, beginner-friendly explanations

LearnPython.org - Loops

https://www.learnpython.org/en/Loops

- Interactive tutorial with exercises
- Practice for loops in your browser
- Complete the exercise at the end

Real Python - For Loops

https://realpython.com/python-for-loop/

- Comprehensive guide to Python for loops
- Detailed examples and explanations
- Read the first half focusing on basic concepts

Understanding range()

W3Schools Python range() Function

https://www.w3schools.com/python/ref_func_range.asp

- Three ways to use range()
- Interactive examples
- Common use cases

Programiz - Python range()

https://www.programiz.com/python-programming/methods/built-in/range

- Clear explanation with examples
- Using range() with for loops
- Step parameter explained

range() Quick Examples

```
# range(stop) - counts from 0
for i in range(5):
```

```
    print(i)  # Prints: 0, 1, 2, 3, 4

# range(start, stop) - counts from start to stop-1
for i in range(2, 7):
    print(i)  # Prints: 2, 3, 4, 5, 6

# range(start, stop, step) - counts by step
for i in range(0, 10, 2):
    print(i)  # Prints: 0, 2, 4, 6, 8

# Counting backwards
for i in range(10, 0, -1):
    print(i)  # Prints: 10, 9, 8, ..., 1
```

Common range() Mistakes

```
# WRONG - range stops BEFORE the end number
for i in range(1, 5):
    print(i)  # Prints 1, 2, 3, 4 (NOT 5!)

# CORRECT - to include 5, use 6 as stop
for i in range(1, 6):
    print(i)  # Prints 1, 2, 3, 4, 5

# WRONG - trying to use range() directly as a list
numbers = range(5)
print(numbers)  # Prints: range(0, 5), not [0, 1, 2, 3, 4]

# CORRECT - convert to list if needed
numbers = list(range(5))
print(numbers)  # Prints: [0, 1, 2, 3, 4]
```

Basic For Loop Patterns

Pattern 1: Process Each Item in a List

```python
# Print each item
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Transform each item
prices = [10, 20, 30]
for price in prices:
    discounted = price * 0.9
    print(f"${price} is now ${discounted}")
```

Pattern 2: Build a New List

```python
# Start with empty list, build it up
numbers = [1, 2, 3, 4, 5]
doubled = []

for num in numbers:
    doubled.append(num * 2)

print(doubled)  # [2, 4, 6, 8, 10]
```

Pattern 3: Accumulate a Total

```python
# Start with 0, add each item
prices = [12.99, 8.50, 15.75]
total = 0

for price in prices:
    total = total + price

print(f"Total: ${total:.2f}")
```

Pattern 4: Count Items That Match

```
# Start with 0, increment when condition is true
scores = [85, 92, 55, 78, 95, 63]
passing = 0

for score in scores:
    if score >= 60:
        passing = passing + 1

print(f"Passing scores: {passing}")
```

Pattern 5: Filter Items

```
# Keep only items that match criteria
numbers = [3, 8, 15, 6, 22, 9]
large_numbers = []

for num in numbers:
    if num > 10:
        large_numbers.append(num)

print(large_numbers)  # [15, 22]
```

Looping with Indices Using range()

GeeksforGeeks - Python range() with len()

https://www.geeksforgeeks.org/python-range-function/

- Using range() with len() to access indices
- When to use indices vs direct iteration
- Multiple examples

Why Use Indices?

```python
# When you need the position number
students = ["Alice", "Bob", "Charlie"]

# Using indices
for i in range(len(students)):
    print(f"{i + 1}. {students[i]}")
# Output:
# 1. Alice
# 2. Bob
# 3. Charlie

# When you need to modify the list
scores = [85, 92, 78]
for i in range(len(scores)):
    scores[i] = scores[i] + 5  # Add 5 bonus points
print(scores)  # [90, 97, 83]

# When working with parallel lists
names = ["Alice", "Bob", "Charlie"]
ages = [20, 19, 21]

for i in range(len(names)):
    print(f"{names[i]} is {ages[i]} years old")
```

Direct Iteration vs Index-Based

```python
fruits = ["apple", "banana", "cherry"]

# Direct iteration - simpler when you just need the item
for fruit in fruits:
    print(fruit)

# Index-based - necessary when you need position or to modify
for i in range(len(fruits)):
    print(f"Fruit {i}: {fruits[i]}")

# Rule of thumb: Use direct iteration unless you specifically need indices
```

Looping Through Strings

String Iteration Examples

```
# Process each character
word = "Python"
for letter in word:
    print(letter)

# Count specific characters
text = "Hello World"
vowel_count = 0

for char in text:
    if char.lower() in "aeiou":
        vowel_count = vowel_count + 1

print(f"Vowels: {vowel_count}")

# Build a new string
original = "hello"
uppercase = ""

for char in original:
    uppercase = uppercase + char.upper()

print(uppercase)  # "HELLO"

# Filter characters
text = "Hello123World"
letters_only = ""

for char in text:
    if char.isalpha():
        letters_only = letters_only + char

print(letters_only)  # "HelloWorld"
```

Practical String Processing

```
# Remove spaces from a string
text = "H e l l o"
no_spaces = ""
```

```
for char in text:
    if char != " ":
        no_spaces = no_spaces + char

print(no_spaces)  # "Hello"

# Count word occurrences in a sentence
sentence = "the cat and the dog"
word_count = 0

for char in sentence:
    if char == " ":
        word_count = word_count + 1

word_count = word_count + 1  # Add 1 for last word
print(f"Words: {word_count}")
```

Nested For Loops

W3Schools - Nested Loops

https://www.w3schools.com/python/gloss_python_for_nested.asp

- Simple nested loop examples
- Understanding how nested loops work
- Interactive examples

Understanding Nested Loops

A nested loop is a loop inside another loop. The inner loop runs completely for each iteration of the outer loop.

How Nested Loops Execute

```
# Simple example to see the pattern
for i in range(3):  # Outer loop: 0, 1, 2
    print(f"Outer loop: i = {i}")
    for j in range(2):  # Inner loop: 0, 1
        print(f"  Inner loop: j = {j}")
    print()  # Blank line
```

```
# Output shows:
# Outer loop: i = 0
#   Inner loop: j = 0
#   Inner loop: j = 1
#
# Outer loop: i = 1
#   Inner loop: j = 0
#   Inner loop: j = 1
#
# Outer loop: i = 2
#   Inner loop: j = 0
#   Inner loop: j = 1
```

Nested Loop Examples

```
# Multiplication table
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i} x {j} = {i * j}")
    print()  # Blank line after each number's table

# Grid of coordinates
for row in range(3):
    for col in range(4):
        print(f"({row},{col})", end=" ")
    print()  # New line after each row

# All combinations
colors = ["red", "blue"]
sizes = ["S", "M", "L"]

for color in colors:
    for size in sizes:
        print(f"{size} {color} shirt")

# Pattern printing
for i in range(1, 6):
    for j in range(i):
        print("*", end="")
    print()  # New line after each row
# Output:
```

```
# *
# **
# ***
# ****
# *****
```

Practical Nested Loop: Comparing Lists

```
# Find matching items between two lists
list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]
matches = []

for item1 in list1:
    for item2 in list2:
        if item1 == item2:
            matches.append(item1)

print(f"Matching items: {matches}")  # [4, 5]
```

Combining Loops with Conditionals

Powerful Combinations

```
# Grade analyzer with categories
scores = [85, 92, 55, 78, 95, 63, 88]

a_grades = []
b_grades = []
c_grades = []
failing = []

for score in scores:
    if score >= 90:
        a_grades.append(score)
    elif score >= 80:
        b_grades.append(score)
    elif score >= 70:
        c_grades.append(score)
```

```
    elif score >= 60:
        failing.append(score)

print(f"A's: {a_grades}")
print(f"B's: {b_grades}")
print(f"C's: {c_grades}")
print(f"Failing: {failing}")

# Find numbers divisible by both 3 and 5
for num in range(1, 51):
    if num % 3 == 0 and num % 5 == 0:
        print(f"{num} is divisible by both 3 and 5")
```

Common Loop Mistakes and Solutions

Mistake 1: Off-by-One Errors with range()

```
# WRONG - misses last item
fruits = ["apple", "banana", "cherry", "date"]
for i in range(len(fruits) - 1):  # Goes 0 to 2, skips index 3
    print(fruits[i])

# CORRECT
for i in range(len(fruits)):  # Goes 0 to 3
    print(fruits[i])

# BETTER - use direct iteration when possible
for fruit in fruits:
    print(fruit)
```

Mistake 2: Forgetting Indentation

```
# WRONG - syntax error
for i in range(5):
print(i)  # Not indented!

# WRONG - logic error
```

```
total = 0
for num in [1, 2, 3, 4, 5]:
    total = total + num
print(f"Sum: {total}")  # This line runs after EACH iteration if indented wrong

# CORRECT
total = 0
for num in [1, 2, 3, 4, 5]:
    total = total + num
print(f"Sum: {total}")  # This line runs AFTER loop completes
```

Mistake 3: Modifying List While Iterating

```
# WRONG - causes problems
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num % 2 == 0:
        numbers.remove(num)  # Don't modify list while looping!

# CORRECT - build new list
numbers = [1, 2, 3, 4, 5]
odd_numbers = []
for num in numbers:
    if num % 2 != 0:
        odd_numbers.append(num)

print(odd_numbers)  # [1, 3, 5]
```

Advanced Patterns

Finding Maximum with Position

```
scores = [85, 92, 78, 95, 88]

# Find highest score and its position
highest = scores[0]
```

```
position = 0

for i in range(len(scores)):
    if scores[i] > highest:
        highest = scores[i]
        position = i

print(f"Highest score: {highest}")
print(f"Position: {position}")
print(f"Found at index {position}")
```

Processing Parallel Lists

```
# Student records using parallel lists
names = ["Alice", "Bob", "Charlie", "Diana"]
scores = [85, 92, 78, 95]
grades = ["B", "A", "C", "A"]

# Print report card
print("Report Card:")
for i in range(len(names)):
    print(f"{names[i]}: {scores[i]} ({grades[i]})")

# Calculate class average
total = 0
for score in scores:
    total = total + score
average = total / len(scores)

print(f"\nClass average: {average:.1f}")
```

Counting Patterns

```
# Count different types of characters
text = "Hello World 123!"

letters = 0
```

```
digits = 0
spaces = 0
others = 0

for char in text:
    if char.isalpha():
        letters = letters + 1
    elif char.isdigit():
        digits = digits + 1
    elif char == " ":
        spaces = spaces + 1
    else:
        others = others + 1

print(f"Letters: {letters}")
print(f"Digits: {digits}")
print(f"Spaces: {spaces}")
print(f"Other characters: {others}")
```

Quick Reference

For Loop Syntax

```
# Loop through list
for item in my_list:
    # use item

# Loop through string
for char in my_string:
    # use char

# Loop with range
for i in range(5):  # 0, 1, 2, 3, 4
    # use i

for i in range(2, 7):  # 2, 3, 4, 5, 6
    # use i

for i in range(0, 10, 2):  # 0, 2, 4, 6, 8
    # use i
```

```
# Loop with indices
for i in range(len(my_list)):
    # use my_list[i] and i

# Nested loop
for i in range(3):
    for j in range(3):
        # runs 9 times total
```

Common Patterns

```
# Accumulate total
total = 0
for num in numbers:
    total = total + num

# Count matches
count = 0
for item in items:
    if condition:
        count = count + 1

# Build new list
new_list = []
for item in old_list:
    new_list.append(item * 2)

# Filter list
filtered = []
for item in items:
    if condition:
        filtered.append(item)

# Find maximum
max_val = list[0]
for item in list:
    if item > max_val:
        max_val = item
```